Sistemas Distribuidos

Ayudantía 5

Profesor Nicolas Hidalgo

Ayudante cátedra: Vicente Díaz, Benjamín Aceituno
Ayudante corrector: Isidora Gonzalez, Natalia Ortega

Contactos

- **Benjamín Aceituno**
  benjamin.aceituno@mail.udp.cl
- **Vicente Díaz**
  Vicente.diaz5@mail.udp.cl
- **Isidora González**
  isidora.gonzalez4@mail.udp.cl
- **Natalia Ortega**
  natalia.ortega1@mail.udp.cl



Contenidos de hoy

01
Chord

Anillo lógico, Finger Table, Join/Leave

02
Pastry

Ruteo por prefijo, Leaf Set, Routing Table, Neighborhood Set

03
P2P No Estructurado

Flooding, BFS, Random Walks, Super-Peers

04
Comparativa

Diferencias clave entre los modelos

CHORD

¿Qué es Chord?

Anillo lógico

Los nodos se organizan en un anillo circular con identificadores del 0 al 2^m.

Identificadores

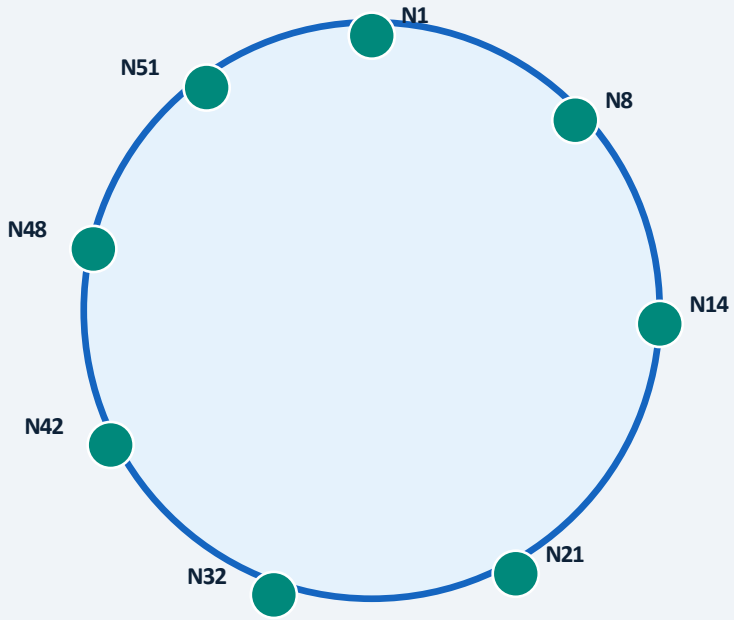
Cada nodo recibe un NodeID = SHA-1(IP:puerto). Los recursos reciben un KeyID = SHA-1(nombre).

Auto-organizado

No necesita coordinación central. El ruteo es de tipo árbol.

Complejidad

Búsqueda en O(log N). Mantenimiento en $O(\log N)^2$.



Anillo Chord

CHORD

Regla del Sucesor & Participación

Regla del Sucesor: un recurso con clave k se asigna al primer nodo cuyo ID ≥ k en sentido horario (unidireccional).

Chord vs Churn

- Churn: Join/Leave crean inconsistencias en table.
- Reparacion es costosa, protocolo de estabilización.

Join (Peer entra)

- El nuevo nodo calcula su ID con SHA-1(IP:puerto).
- Contacta al sucesor de su identificador.
- El sucesor entrega información de su antecesor.
- El nuevo nodo se registra como antecesor del successor.
- Se hace responsible de un grupo de recursos (sucesor).

Leave (Peer deja la red)

- El nodo entrega todos sus recursos al sucesor.
- Notifica al sucesor y al antecesor del cambio.
- Se elimina del anillo sin interrumpir el servicio.

Churn (Dinámica)

- Join: nuevo nodo hereda claves a su sucesor.
- Leave: transfiere claves antes de irse.
- Protocolo de estabilización periódica.
- Detección de caídas con heartbeats.
- Guarda réplicas en S sucesores para tolerancia a fallos.

CHORD

Finger Table: búsqueda eficiente

Clave
Cada salto acerca al destino exponencialmente.

→ **Máximo $\log_2 N$ saltos por consulta.**
→ Tabla de solo m entradas por nodo.

Sin Finger Table

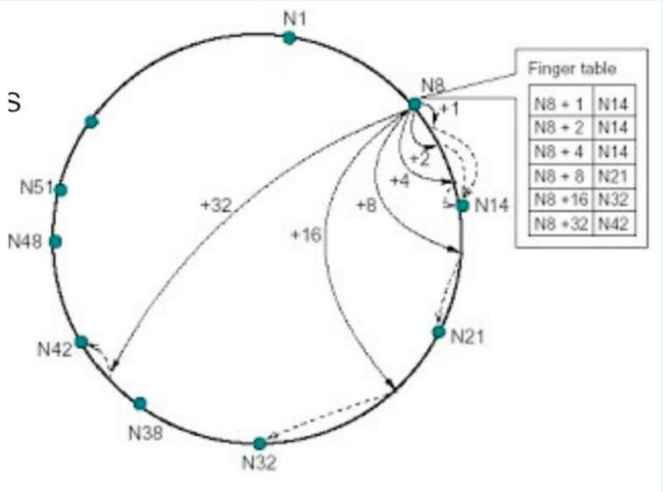
Búsqueda secuencial nodo a nodo → O(N) en el peor caso.
Muy lento con miles de nodos.

Con Finger Table

Saltos exponenciales: entrada i apunta al nodo a 2^(i−1) posiciones adelante → O(log N).

Ejemplo – Finger Table del nodo N8 (m = 6 bits):

Entrada i	Buscar (N8 + 2^i−1) mod 2^6	Nodo sucesor
i = 1	N8 + 1 = 9	N14
i = 2	N8 + 2 = 10	N14
i = 3	N8 + 4 = 12	N14
i = 4	N8 + 8 = 16	N21
i = 5	N8 + 16 = 24	N32
i = 6	N8 + 32 = 40	N42



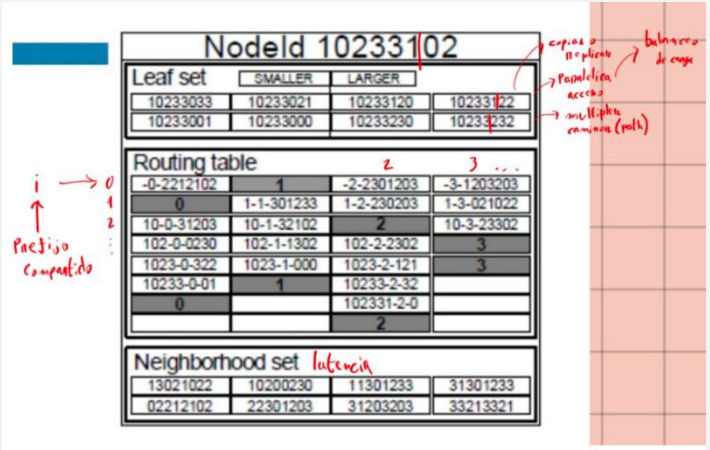
PASTRY

¿Qué es Pastry?

Pastry es un sistema P2P estructurado con ruteo basado en prefijos. Cada nodo tiene un ID único de 128 bits (SHA-1). El enrutamiento logra el destino en O(log N) pasos, sin depender de un servidor central.

Diferencias clave respecto a Chord:

Aspecto	Chord	Pastry
Identificadores	128 bits (SHA-1) → anillo unidireccional	128 bits (SHA-1) → anillo bidireccional
Ruteo	Finger Table (saltos 2^i)	Routing Table por prefijo compartido
Cercanía	Solo distancia lógica en el anillo	Considera latencia real (Neighborhood Set)
Tolerancia	Lista de S sucesores	Leaf Set garantiza entrega final



PASTRY

Las 3 estructuras de cada nodo

Routing Table

- Estructura principal de enrutamiento.
- Filas indexadas por tamaño de prefijo compartido con el nodo actual.
- Cada fila tiene entradas para todos los posibles dígitos siguientes.
- Cada salto incrementa el prefijo común con la clave destino.
- → Complejidad O(log N).

Leaf Set

- Contiene los nodos numéricamente más cercanos (mayores y menores).
- Se usa para los últimos pasos del enrutamiento.
- Garantiza la entrega final del mensaje.
- Permite replicación y balance de carga.
- → Multi-path search y tolerancia a fallos.

Neighborhood Set

- Guarda nodos físicamente cercanos (por latencia o IP).
- No se usa directamente para enrutar mensajes.
- Optimiza la eficiencia real de la red.
- Permite generar búsquedas diversas en caminos.
- → Localidad de red (ventaja sobre Chord).

Regla de búsqueda: si la clave está en el Leaf Set → entrega directa. Si no → usar Routing Table para acercar prefijo.

PASTRY

Cómo funciona el enrutamiento

1. **Un nodo quiere enviar un mensaje a la clave d46a1c.**

   Primero busca en su Leaf Set si hay un nodo con ID cercano.

2. **No está en el Leaf Set → consulta la Routing Table.**

   Busca la fila cuyo prefijo coincida más con d46a1c.

3. **Elige el nodo que comparte más dígitos de prefijo con la clave.**

   En cada salto, el prefijo común crece en al menos un dígito.

4. **El proceso se repite en el siguiente nodo.**

   Cada salto acerca al destino hasta que el prefijo es completo.

5. **Llegada: el nodo responsable entrega o procesa el mensaje.**

   Garantizado por el Leaf Set del nodo más cercano al destino.

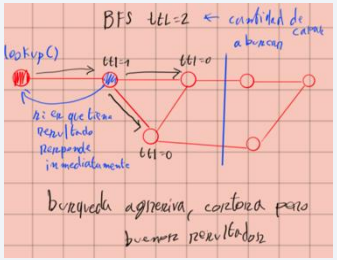
P2P NO ESTRUCTURADO

Cuando la libertad importa más que la eficiencia

- Estructura aleatoria – sin mapeo hash de datos
- Conocimiento local – cada nodo conoce solo sus vecinos
- Búsqueda por inundación (Flooding)
- Gran autonomía y descentralización
- Efectivo para contenido popular

P2P NO ESTRUCTURADO

Métodos de búsqueda



Flooding

- Mensaje a TODOS los vecinos conocidos.
- TTL limita la propagación.
- Poco eficiente, no escala.
- Calidad de resultados vs # mensajes.

BFS (Breadth-First Search)

- Flooding controlado por TTL o routing hops.
- Forward de consulta si no hay resultado.
- Alta redundancia, mensajes duplicados.

Iterative Deepening

- Múltiples BFS secuenciales con TTL creciente.
- Comienza poco profundo, itera si no encuentra.
- Caching en nodos del camino para evitar repetidos.

Directed BFS

- Elige subconjunto de vecinos más prometedores.
- Heurísticas: menos saltos, más mensajes recibidos, menor latencia.

Local Indexes

- Conoce archivos de vecinos y vecinos de vecinos.
- Mejora eficiencia pero usa mucha memoria.
- Poco útil con alto churn.

Random Walks / K-RW

- Mensaje a UN vecino aleatorio.
- K-Random Walks: K búsquedas paralelas.
- Reduce mensajes, aumenta latencia vs BFS.

P2P NO ESTRUCTURADO

Super-Peers: jerarquía en P2P

Super-Peers: nodos más potentes que actúan como "miniservidores". Rompen la simetría del overlay, introduciendo una organización jerárquica.

¿Qué los hace especiales?

- Mayor capacidad de procesamiento
- Mayor ancho de banda
- Mayor estabilidad en la red
- Asumen roles especiales de índice y ruteo

Ventajas

- Reducen el flooding en la red
- Mejoran la eficiencia de búsqueda
- Permiten anonimato opcional (FileSharing)

Proceso de elección

- Cualquier nodo que cumpla las condiciones puede ser SP
- Algoritmos de elección dinámica
- Las aplicaciones imponen sus propios criterios

Desventajas

- No son estables (costoso mantenerlos)
- Pesadilla para overlays estructurados
- Entre más estable el SP, mejor el sistema

Comparativa general

Criterio	Chord	Pastry	P2P No Estructurado
Organización	Anillo unidireccional	Anillo bidireccional	Grafo aleatorio
Identificación	SHA-1 (NodeID / KeyID)	SHA-1 (128 bits)	No requerida
Búsqueda	O(log N) – Finger Table	O(log N) – Routing Table	O(N) – Flooding/BFS/RW
Localidad física	No considera latencia	Sí – Neighborhood Set	Parcial – Super-Peers
Tolerancia a fallos	Lista de S sucesores	Leaf Set + replicación	Depende del método
Búsquedas complejas	Solo exactas (KeyID)	Solo exactas (KeyID)	Comparación de strings
Churn (Join/Leave)	Estabilización periódica	Estabilización periódica	Alta autonomía
Escalabilidad	Alta (log N)	Alta (log N)	Baja (flooding)